Medición y regla de Born

Genesis Silesky

20 de octubre de 2026

1. Medición y regla de Born

1.1. Objetivo

Comprobar experimentalmente que $P(0) = |\alpha|^2$ y $P(1) = |\beta|^2$, y ver cómo la frecuencia observada converge a esas probabilidades al aumentar los *shots*.

1.2. Conceptos previos

Concepto previo: regla de Born, colapso y frecuencia relativa

Ver sección **??**. Dos ideas: (i) medir produce un resultado clásico aleatorio con las probabilidades de Born; (ii) tras medir, el estado *colapsa* al resultado observado, y la superposición original se pierde.

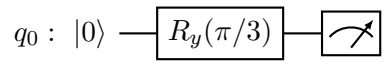
1.3. Fundamento teórico

Para $|\psi\rangle = \alpha\,|0\rangle + \beta\,|1\rangle$, la medición en la base computacional da 0 con probabilidad $|\alpha|^2$ y 1 con $|\beta|^2$. No se puede conocer α, β con una sola medición; hay que repetir el experimento N veces (cada repetición es un *shot*). La frecuencia $\hat{p}$ de "1" tiene error típico

$$\sigma = \sqrt{\frac{p(1-p)}{N}}. \tag{1}$$

Para $p = 0.25$: $\sigma \approx 0.137$ ($N = 10$), 0.043 ($N = 100$), 0.014 ($N = 1000$), 0.0043 ($N = 10000$).

Preparación del estado. La rotación $R_y(\theta)\,|0\rangle = \cos\frac{\theta}{2}\,|0\rangle + \sin\frac{\theta}{2}\,|1\rangle$. Para que $P(1) = \sin^2\frac{\theta}{2} = 0.25$ se necesita $\theta = 2\arcsin(0.5) = \pi/3$.



1.4. Código (completamente comentado)

```python
# %% [markdown]
# # 04 - Medicion: P(0) = |alpha|^2 y P(1) = |beta|^2

# %% Celda 1: importaciones
import sys

import numpy as np
from qiskit import QuantumCircuit
from qiskit_aer import AerSimulator  # simulador local de circuitos
```

```python
from qiskit.visualization import plot_histogram

# La consola de Windows (cp1252) no imprime el dibujo del circuito sin esto.
if hasattr(sys.stdout, "reconfigure"):
    sys.stdout.reconfigure(encoding="utf-8")

# %% Celda 2: preparar un qubit con P(1) = 0.25
# La rotacion Ry(theta) lleva |0> a cos(theta/2)|0> + sin(theta/2)|1>.
# Queremos sin^2(theta/2) = 0.25, es decir theta = 2*arcsin(sqrt(0.25)).
theta = 2 * np.arcsin(np.sqrt(0.25))

# QuantumCircuit(1, 1): 1 qubit y 1 bit clasico donde guardar la medicion.
qc = QuantumCircuit(1, 1)
qc.ry(theta, 0)      # prepara el estado con las amplitudes deseadas
qc.measure(0, 0)     # mide el qubit 0 y guarda el resultado en el bit clasico 0
print(qc.draw())     # dibujo del circuito en texto

# %% Celda 3: repetir la medicion (shots)
# Una sola medicion da un unico 0 o 1 y destruye la superposicion.
# Para estimar las probabilidades hay que repetir el experimento muchas
# veces: cada repeticion se llama "shot".
sim = AerSimulator()
for shots in [10, 100, 1000, 10000]:
    # run() lanza el circuito; result().get_counts() devuelve cuantas veces
    # salio cada resultado, por ejemplo {'0': 749, '1': 251}.
    counts = sim.run(qc, shots=shots).result().get_counts()
    print(shots, "shots ->", counts, f"| freq(1) = {counts.get('1', 0) / shots:.3f}")
# Ley de los grandes numeros: con mas shots, freq(1) se acerca a 0.25.

# %% Celda 4: histograma
# En Colab o Jupyter la figura se muestra sola; en terminal se puede
# guardar con plot_histogram(...).savefig("hist.png").
plot_histogram(sim.run(qc, shots=1000).result().get_counts())
```

1.5. Explicación celda por celda

Celda 1 Además de las importaciones, reconfigura la salida estándar a UTF-8: sin eso la consola de Windows no puede imprimir el dibujo del circuito (error `UnicodeEncodeError`).

Celda 2 Calcula θ, crea un circuito con 1 qubit y 1 bit clásico, aplica R_y y mide.

Celda 3 Ejecuta $10, 100, 1000, 10000$ shots y calcula la frecuencia de "1". `counts.get('1', 0)` evita un error si "1" nunca salió.

Celda 4 Dibuja el histograma (en script puede guardarse con `savefig`).

1.6. Salida obtenida

Los números cambian en cada ejecución

La medición es aleatoria: los conteos exactos varían. Lo que debe reproducirse es la *tendencia*: la frecuencia de "1" se acerca a 0.25 al aumentar los shots.

```
10 shots -> {'1': 3, '0': 7} | freq(1) = 0.300
100 shots -> {'0': 73, '1': 27} | freq(1) = 0.270
1000 shots -> {'0': 728, '1': 272} | freq(1) = 0.272
10000 shots -> {'0': 7501, '1': 2499} | freq(1) = 0.250
```

1.7. Interpretación

Con 10 shots la frecuencia puede alejarse mucho de 0.25 (error típico ≈ 0.14); con 10 000 queda cerca (error ≈ 0.004). La incertidumbre estadística disminuye como $1/\sqrt{N}$.

1.8. Ejercicios

1. Prepare un qubit con $P(1) = 0.9$ y mida 2000 shots.
2. Repita 20 veces la corrida de 100 shots y grafique la dispersión de la frecuencia; compárela con $\sigma = \sqrt{p(1-p)/N}$.
3. ¿Qué obtiene si mide dos veces seguidas el mismo qubit? (Pista: colapso.)